SIMATS ENGINEERING

Short Answer Test

NAME: Charuthya P
REG NO: 192421148
COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*
Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3

7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3

18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3
19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3

30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

1. Testing as an Engineering Activity

Testing should be treated as a systematic engineering activity rather than random execution of test cases. The team should understand the requirements, identify test conditions, design test cases, prepare test data, execute tests, record defects, and perform regression testing.

Structured test design improves quality because important functional, boundary, and invalid conditions are covered systematically. For example, a login page can be tested with valid credentials, invalid username, invalid password, blank fields, and boundary inputs. This improves defect detection and reduces missed defects.

2. Well-Defined Testing Process

A well-defined testing process provides a standard sequence for planning, designing, executing, reporting, and closing testing activities.

In a banking system, testers can prepare test cases for login, balance checking, deposits, withdrawals, and fund transfers. Defects are recorded, fixed, retested, and included in regression testing.

This improves **consistency, reliability, traceability, and defect detection**.

3. Testing as a Structured Process in SDLC

Testing should be performed throughout the Software Development Life Cycle rather than only after coding.

- **Requirement phase:** Review requirements for ambiguity.
- **Design phase:** Identify risks and test conditions.
- **Coding phase:** Perform unit testing.
- **Integration phase:** Test communication between modules.
- **System testing:** Test the complete application.
- **Acceptance testing:** Verify user requirements.
- **Regression testing:** Retest after changes.

Following a structured process reduces defects and improves software quality.

4. Error, Defect and Failure

An **error** is a human mistake made during software development.

A **defect** is a flaw in the software caused by an error.

A **failure** occurs when the software executes the defective code and produces an incorrect result.

Example: A programmer writes an incorrect password condition. This is a defect. If a valid user is rejected during login, it is a failure.

Thus:

Error → Defect → Failure

5. Early Defect Detection

The principle of early testing should be applied. Testing should begin as early as possible in the SDLC.

Testers can review requirements and design documents before coding begins. For example, if an ambiguous payment requirement is identified during requirement review, it can be corrected before implementation.

Early defect detection:

- Reduces development cost
- Reduces rework
- Saves time
- Prevents defects from reaching later stages
- Improves software quality

6. Tester Involvement in Requirement Phase

A tester should participate in requirement analysis by checking requirements for:

- Correctness
- Completeness
- Consistency
- Clarity
- Testability

For example, in a healthcare system, the tester can identify unclear requirements related to patient records or access permissions.

Early tester involvement prevents misunderstandings from becoming implementation defects and reduces the cost of fixing problems later.

7. Defects from Unclear Requirements

Defects can originate when requirements are incomplete, ambiguous, inconsistent, or misunderstood.

For example, if the overtime calculation rule in a payroll system is unclear, developers may implement it incorrectly, resulting in incorrect salary calculations.

Requirement clarity can be improved through:

- Requirement reviews
- Precise specifications
- Examples
- Acceptance criteria
- Communication with stakeholders

Clear requirements prevent many defects before coding begins.

8. Boundary Value Defect

A system crash at boundary values is generally a **boundary-related defect** caused by insufficient boundary testing.

Boundary Value Analysis (BVA) tests values around the limits.

For example, if an age field accepts **18 to 60**, the tester should test:

17, 18, 19, 59, 60, 61

Testing these values can identify defects at the minimum and maximum limits and prevent failures.

9. Defect Repository

A defect repository stores information about previously detected defects, such as:

- Defect ID
- Description
- Module
- Severity
- Priority
- Status
- Cause
- Resolution

Testers can study historical defects and create new test cases based on repeated problems.

For example, if payment validation defects repeatedly occur, similar validation cases should be added to future regression testing.

This reduces repeated defects and improves software quality.

10. Functional Defect in ATM

An ATM withdrawal problem caused by incorrect transaction logic is a functional defect because the system does not perform its required function correctly.

For example, the ATM may deduct money from the account but fail to dispense cash.

It can be prevented by testing:

- Valid withdrawals
- Insufficient balance
- Withdrawal limits
- Account balance updates
- Transaction failures
- Error handling

Integration and system testing should verify the complete transaction.

11. Defect Clustering

Defect clustering means that a small number of modules often contain a large proportion of defects.

Testers should analyze defect history and identify modules with high defect concentration.

For example, if most defects occur in the payment and authentication modules, these modules should receive more testing effort.

This helps:

- Prioritize testing
- Use resources efficiently
- Focus on high-risk modules
- Improve defect detection

12. Performance Defect

Slow performance under heavy user load is a **non-functional performance defect**.

It may be caused by:

- Inefficient algorithms
- Database bottlenecks
- Insufficient resources
- Poor scalability

The defect can be identified through **load testing, stress testing, and performance testing**.

Proper classification helps testers identify the cause and select the appropriate solution.

13. Historical Defect Data for Regression Testing

Historical defect data can be reused to improve regression testing.

Testers should identify:

- Previously failed scenarios
- Frequently affected modules
- Repeated defects
- Defects caused by previous changes

These cases can be added to the regression test suite.

For example, if a payment calculation defect occurred earlier, the same test should be executed after future changes to payment logic.

This improves coverage and saves test-design effort.

14. Tester Role in Integration Defects

Testers help minimize integration defects by:

- Understanding module interfaces
- Identifying integration risks
- Preparing integration test cases
- Participating in reviews
- Communicating with developers

For example, testers can verify whether the correct data is transferred from a payment module to an account module.

Early communication reduces misunderstandings and helps detect integration defects sooner.

15. Defect Prevention

Defect prevention focuses on preventing defects instead of only finding them after implementation.

The team should:

- Analyze root causes
- Review requirements
- Follow coding standards
- Conduct reviews
- Use checklists
- Learn from previous defects

If the same validation defect occurs repeatedly, the team should identify its root cause and improve the development and testing process.

Defect prevention improves long-term software quality.

16. Standardized Testing Strategy

A mobile application should use a structured and standardized testing strategy.

The team should define:

- Common testing procedures
- Test-case formats
- Test data standards
- Defect-reporting procedures
- Entry and exit criteria
- Regression test suites

The application should also be tested for functionality, compatibility, usability, and performance.

Standardization ensures consistency between testers and improves application reliability.

17. Testing vs Debugging

Testing is the process of evaluating software to identify failures and defects.

Debugging is the process of finding the cause of a defect and correcting the code.

Example: If an ATM gives an incorrect balance after withdrawal, testing identifies and reports the problem. The developer then debugs the transaction logic, finds the incorrect calculation, and fixes it.

Therefore:

Testing → Finds the problem

Debugging → Finds and fixes the cause

18. Cross-Browser Testing

A cross-browser testing strategy should test the web application on supported browsers such as Chrome, Edge, Firefox, and Safari.

Testers should check:

- Functionality
- User interface
- CSS rendering
- JavaScript behavior
- Responsive design
- Browser compatibility

Automated browser testing can improve coverage.

Cross-browser testing ensures that users get consistent application behavior across different browsers.

19. Proper Validation

Improper validation may allow incorrect or inconsistent data to enter the system.

Testers should check:

- Valid inputs
- Invalid inputs
- Boundary values
- Blank values
- Incorrect formats
- Unexpected inputs

For example, in a logistics application, shipment quantity, destination, and delivery date should be validated.

Proper validation detects problems early and improves system reliability.

20. Systematic Test Case Design

Random testing may miss important conditions because it does not provide systematic coverage.

Testers should use techniques such as:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- Requirement-based testing

The team should identify test conditions, prepare expected results, execute tests, and document defects.

Systematic test design provides better coverage and improves defect detection.

21. Data Consistency Between Modules

Data inconsistency can occur because of:

- Incorrect interfaces
- Data mapping errors
- Synchronization problems
- Incompatible data formats

Testers should trace data from one module to another and compare the source and destination values.

Integration testing should include valid data, invalid data, missing fields, updates, and error conditions.

Proper integration testing ensures that data remains consistent across modules.

22. Tester Role Across SDLC

Testers should participate throughout the SDLC.

- **Requirements:** Review clarity and testability.
- **Design:** Identify risks and test conditions.
- **Development:** Support unit and integration testing.
- **System testing:** Execute functional and non-functional tests.
- **Acceptance:** Verify user requirements.
- **Maintenance:** Perform regression testing.

Early tester involvement improves communication, detects defects sooner, and reduces rework.

23. Test Coverage for Peak Usage

Failure during peak usage indicates that important scenarios were not adequately tested.

Testers should perform:

- Load testing
- Stress testing
- Performance testing
- Concurrent-user testing

Scenarios should include maximum users, high transaction volume, concurrent requests, and resource limits.

Better coverage of peak-use conditions reduces production failures.

24. Defect Documentation and Repository

A defect repository should record:

- Defect ID
- Title
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Environment
- Status
- Resolution

Proper documentation makes defects easy to track from discovery to closure.

It also supports trend analysis, root-cause analysis, and regression-test design.

25. Structured Testing for Banking System

A banking system should follow a structured testing process covering:

1. Requirement analysis
2. Test planning
3. Test-case design
4. Test execution
5. Defect reporting
6. Retesting
7. Regression testing

Important transactions such as deposits, withdrawals, transfers, balance updates, authentication, and error handling should be tested.

Structured testing improves coverage, detects defects early, and increases banking-system reliability.

26. Edge Case and Boundary Testing

Testing only normal inputs is not sufficient because defects often occur at boundaries and unusual conditions.

Testers should include:

- Minimum values
- Maximum values
- Just-inside values
- Just-outside values
- Empty values
- Zero
- Null values
- Unexpected values

For example, if a quantity range is **1–100**, test:

0, 1, 2, 99, 100, 101

Boundary testing improves defect detection.

27. Process-Based Testing

A process-based testing approach should define standard:

- Testing activities
- Responsibilities
- Test-case formats
- Review procedures
- Defect-reporting rules
- Entry and exit criteria

The same quality standards should be followed across modules.

Standardization reduces inconsistency, improves traceability, and ensures that important testing activities are not skipped.

28. Unexpected Input Validation

The web application should validate unexpected inputs before processing them.

Testers should check:

- Invalid data types
- Special characters
- Extreme values
- Blank values
- Malformed input
- Unexpected combinations

The system should reject invalid data safely and provide suitable error messages without crashing.

Proper input validation improves robustness and reliability.

29. Reducing Defect Leakage

Frequent defect leakage indicates gaps in requirements, test coverage, test design, or regression testing.

The team should:

- Analyze production defects
- Perform root-cause analysis
- Identify missing test cases
- Improve regression testing
- Review requirements
- Focus on high-risk modules

Production defects should be converted into regression test cases so that the same problems do not occur again.

This reduces defect leakage and improves software quality.

30. Early Defect Detection and Efficiency

The testing strategy should emphasize **early and continuous testing**.

Requirements and design should be reviewed before implementation. Developers should perform unit testing, while testers should begin test planning early. Integration and system testing should then be performed systematically.

Early defect detection:

- Reduces rework
- Saves time
- Reduces cost
- Prevents defect propagation
- Improves software quality
- Increases project efficiency

Therefore, testing throughout the SDLC is more effective than testing only at the end.

Code of Conduct:

I Charu*thya P (192421148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.
Charuthya P
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand